PYCON US 2026 · WORKSHOP

Practical Software Testing with Python

Paul Zuradzki · 210 min / 3.5 hours · pytest + unittest

Slides: paulzuradzki.com/talks/pycon-us-2026-tutorial-software-testing
Code: github.com/paulzuradzki/software-testing-workshop-pycon-us-2026

About your instructor

- Paul Zuradzki — Data Science Engineer at Turquoise Health
- Increasing price transparency and simplifying healthcare transactions
- Backend web dev (Django), data pipelines (Airflow), data wrangling (pandas, polars, SQL)
- paulzuradzki.com/about

Where else to find me on the web

- LinkedIn: paulzuradzki
- Mastodon: @paulzuradzki@mastodon.social
- GitHub: @paulzuradzki
- YouTube: @paulzuradzki

PART 0 · 5 MIN

Welcome

Why we test. Today's plan. How we'll work together.

Quick brainstorm

Jot down somewhere you'll see it (paper, doc, sticky note):

1. One function or test you've struggled with at work.
2. What brought you here today.

We come back to these throughout the afternoon.

Why testing matters

- **Correctness:** catch bugs before users do.
- **Confidence:** change code without holding your breath.
- **Design:** testable code is usually well-structured code.
- **AI-assisted dev:** tests give agents the feedback loop they need to self-correct.

Today's plan

Time (PT)	Part	Topic	Min
1:30–2:25	1	Fundamentals	55
2:25–2:30	-	stretch break	5
2:30–3:00	2a	Unit/integration/E2E, parameterization	30
3:00–3:30	-	coffee break	30
3:30–4:05	2b	Parameterization, fixtures	35
4:05–4:45	3	Design, DI & seams, TDD	40
4:45–4:50	-	survey	5
4:50–5:00	-	Wrap-up & Q&A	10

DI = dependency injection. Pass dependencies in as arguments instead of constructing them inside a function. Covered in Part 3. · **E2E** = end-to-end. A test that exercises the full system through real inputs and outputs. Covered in Part 2.

How we'll work

- **Type code by hand.** slow reps matter.
  - See `student/README.md:Instructor Notes on AI Usage`
  - I recommend turning off AI-assist autocompletions
  - I've added some learner-oriented prompts to the repo if you'd like to try AI Q&A
- **Pair when stuck.** Your neighbor probably hit the same wall.
- **Side quests** are optional. Pick one if you finish early.

What this workshop focuses on

- **Hands-on reps.** You'll write tests by hand to build judgment for reading and evaluating tests written by others.
- **Design over tooling.** The last third is about how testable code is shaped, with dependency injection and seams as the worked example.
- **Pragmatic code.** We focus on techniques that come up often in real codebases: fixtures and parameterization.
  - Heavier mocking and advanced fixtures (for a specific database or cloud service, for example) get pointer slides only and would fit a follow-up workshop.
  - The Part 3 design module aims to leave you with the foundations to handle those cases on your own.
  - This is the most unique part of the workshop compared to what you might see in other intros.

PART 1 · 55 MIN

Fundamentals

Setup. First tests. Anatomy. Targeting. Structure.

Writing your first test

```python
# temperature.py
def c_to_f(c: float) -> float:
    return c * 9 / 5 + 32

# test_temperature.py
def test_freezing_point():
    assert c_to_f(0) == 32
```

Run with: `pytest test_temperature.py`

Exercise 1.1 · Warm-up

Open: `modules/1.1_warm_up_temp_script/`

8 min. Test `convert_f_to_c`. Run the script with `python` and with `pytest`.

`unittest` vs `pytest`

```python
# unittest style
import unittest
class TestC2F(unittest.TestCase):
    def test_freezing(self):
        self.assertEqual(c_to_f(0), 32)

# pytest style
def test_freezing():
    assert c_to_f(0) == 32
```

`unittest` ships with Python and uses classes. `pytest` uses plain functions and bare `assert`. Both are valid. We use `pytest` from here on.

Exercise 1.2 · unittest vs pytest

Open: `modules/1.2_unittest_and_pytest/`

10 min. Write the same test twice, once per style. Run each with its matching runner.

Anatomy: Arrange-Act-Assert

```
def test_freezing_point():
    # Arrange
    celsius = 0

    # Act
    fahrenheit = c_to_f(celsius)

    # Assert
    assert fahrenheit == 32
```

The AAA structure makes each test's intent obvious and keeps each test focused on one behavior.

AAA with an object under test

```python
def test_cart_total_includes_tax():
    # Arrange: build the instance and put it in a known state
    cart = ShoppingCart(tax_rate=0.10)
    cart.add(Item("apple", 1.00))
    cart.add(Item("bread", 3.50))

    # Act: invoke the behavior we're checking
    total = cart.total()

    # Assert: one observable outcome
    assert total == 4.95
```

For class-based code, **Arrange** is usually `__init__` plus the calls that set up the state you need before the action.

Exercise 1.3 · Restructure to AAA

Open: `modules/1.3_structuring_a_test/`

8 min. Take an existing test and refactor it into Arrange-Act-Assert form.

Test discovery

Pytest finds tests by walking the current directory and its descendants. The default rules:

- Files named `test_*.py` or `*_test.py`.
- Functions named `test_*`.
- Methods named `test_*` inside classes named `Test*` (no `__init__`).

Files in a `conftest.py`, `__init__.py`, or a class without a `Test*` prefix are silently skipped. If a test you wrote is not running, this is usually why.

REFERENCES

- docs.pytest.org · Test discovery

Demo 1.4 · Targeting

Open: `modules/1.4_targeting/`

~7 min

We'll target a file, a class, a function, a `-k` pattern, and a custom mark, then bump verbosity.

Targeting tests

Which tests do you think will run on each invocation?

```
pytest tests/test_temp.py            # file
pytest tests/test_temp.py::TestC2F   # class
pytest -k "freezing"                  # pattern
pytest -m slow                        # mark
pytest -v                             # verbose
```

Exercise 1.5 · Package structure

Open: `modules/1.5_package_structure/`

10 min. Put source in `src/` and tests in `test/`. Make imports work via the scaffolded `pyproject.toml`.

Source vs. tests structure

```
project/
├── src/
│   └── mypkg/__init__.py
├── tests/
│   └── test_mypkg.py
└── pyproject.toml
```

Tests live next to the code they cover but stay out of the distributed package.

How Python finds your code

When `import mypkg` runs, Python walks `sys.path`: the script's directory, `PYTHONPATH`, then installed site-packages.

`pip install -e .` (or `uv pip install -e .`) registers `src/mypkg` on `sys.path` so both your code and `pytest` can import it the same way the world will.

REFERENCES

- docs.python.org · The module search path
- paulzuradzki.com · Python packaging resources

Stretch break

Stand up. Refill water.

Optional: switch seats. A new neighbor for Part 2 is a fresh pair for spot-checks and pair discussions.

Test Tools & Isolation

Unit vs integration vs e2e. Parameterization. Fixtures. Flakiness.

The testing pyramid

- **Unit:** fast, isolated, many. One function or class.
- **Integration:** multiple units cooperating. Slower, fewer.
- **End-to-end:** full system, real I/O. Slowest, fewest.

Most projects need many fast unit tests, fewer integration tests, and a small number of end-to-end tests.

Exercise 2.1 · Unit test ToDoTracker

Open: `modules/2.1_unit_test_todotracker/`

10 min. Write a unit test for an untested method on `ToDoTracker`.

Exercise 2.2 · Integration test ToDoTracker

Open: `modules/2.2_integration_test_todotracker/`

10 min. Write a test that calls multiple `ToDoTracker` methods and asserts on combined behavior or state.

Parameterized tests

```python
import pytest

@pytest.mark.parametrize("celsius, fahrenheit", [
    (0,    32),
    (100,  212),
    (-40, -40),
    (37,   98.6),
])
def test_c_to_f(celsius, fahrenheit):
    assert c_to_f(celsius) == fahrenheit
```

Each row runs as its own case and reports independently in the failure log.

unittest equivalent: wrap the loop body in `with self.subTest(celsius=celsius):` to get the same per-row reporting. `parametrize` is the pytest idiom; both solve the same problem.

Exercise 2.3 · Parameterized table test

Open: `modules/2.3_parameterized_tests/`

12 min. Convert the temperature converter test into a table test using `pytest.mark.parametrize`.

Edge cases worth thinking about

Two questions for any function:

- **What should always hold?** Properties that survive any input. Output type. Length bounds. Round-trip equality. Idempotence.
- **Which examples are worth picking?** Boundary values (0, -1, max), empty inputs, duplicates, off-by-one neighbours, malformed input.

Postel's robustness principle: be liberal in what you accept, strict in what you produce. Tests at the boundaries protect both halves.

Fixtures: setup & teardown

```python
import pytest, tempfile, pathlib

@pytest.fixture
def tmp_log():
    with tempfile.NamedTemporaryFile(suffix=".log", delete=False) as f:
        path = pathlib.Path(f.name)
    yield path
    path.unlink(missing_ok=True)

def test_writes_to_log(tmp_log):
    tmp_log.write_text("hello")
    assert tmp_log.read_text() == "hello"
```

`yield` separates setup from teardown. Pytest runs the teardown even if the test fails.

Exercise 2.4 · File fixture

Open: `modules/2.4_fixtures/`

15 min. Build a file fixture with `yield` and `tmp_path`. Use it in a test that writes and reads.

Demo · setUp / tearDown vs pytest fixture

5 min. The same scenario in unittest and in pytest, side by side.

```
# unittest
class TestTodo(unittest.TestCase):
    def setUp(self):    self.path = make_temp()
    def tearDown(self): self.path.unlink(missing_ok=True)

# pytest
@pytest.fixture
def todo_path(tmp_path):
    yield tmp_path / "todo.txt"
```

Same lifecycle. The pytest version composes: any test can request `todo_path` by name. `setUp` runs for every method in the class.

Demo · SQLite in-memory fixture

5 min. The fixture pattern from before, applied to a SQLite database.

```python
@pytest.fixture
def db():
    conn = sqlite3.connect(":memory:")
    conn.execute("CREATE TABLE todo (id INTEGER, text TEXT)")
    yield conn
    conn.close()

def test_insert_then_read(db):
    db.execute("INSERT INTO todo VALUES (1, 'buy milk')")
    rows = db.execute("SELECT text FROM todo").fetchall()
    assert rows == [("buy milk",)]
```

`:memory:` databases vanish on close. Every test gets a fresh one.

REFERENCES

- paulzuradzki.com · Unit testing SQL

Common flakiness patterns

- **Order dependence:** tests sharing global state.
- **Time and clocks:** `datetime.now()` in assertions.
- **Filesystem leftovers:** clean up in teardown.
- **Race conditions:** concurrent access without locks.

Hands-on side quests (after 2.4): `modules/side_quests/flakiness_cleanup/` and `modules/side_quests/flakiness_race/`.

Demo · Coverage as a tool

5 min, instructor-led.

```
pytest --cov=src --cov-report=term-missing
```

`term-missing` prints which lines were not exercised. Useful for spotting branches you forgot to test.

```
Name            Stmts   Miss  Cover   Missing
src/discount.py    14      2    86%   23-24
```

Coverage shows you which lines didn't run. A high number doesn't mean the assertions are good. 100% coverage with shallow assertions is still a hole. We see this again in Part 3 with snapshot tests.

REFERENCES

- pytest-cov.readthedocs.io

PART 3 · 40 MIN

Design for Testability

Black-box vs glass-box. DI and seams. TDD-lite. Snapshot testing.

Black-box vs. glass-box

- **Black-box:** test through the public interface only.
- **Glass-box:** test private internals. Coupled to implementation.

Default to black-box so refactors don't break tests when behavior is unchanged.

Pure functions are easiest to test black-box. **Impure** functions touch the world. The DI section that follows is mostly about turning impure functions into testable ones.

Designing for testability

Testable code = code where you can substitute real dependencies for test ones. The thing that lets you substitute is called a **seam**.

Dependency injection is one way to put a seam in your code. The key idea: **separate creation from use**.

Aside: "good design" here means easy to change, easy to test, easy to understand. Long form on the blog.

REFERENCE

- paulzuradzki.com · Dependency injection for Pythonistas

Spot the dependencies

```python
def extract():
    s3_client = boto3.client("s3")            # creates a real S3 client
    bucket_name = os.environ["AWS_BUCKET"]    # reads env

    with NamedTemporaryFile(delete=False) as tmp_file:
        s3_client.download_fileobj(           # network
            Bucket=bucket_name, Key="users.csv", Fileobj=tmp_file
        )
        tmp_file.seek(0)
        with open(tmp_file.name, "r") as f:   # disk
            csv_reader = csv.DictReader(f)
            data = [row for row in csv_reader]

    return data
```

Implicit dependencies: `boto3`, env var, the filesystem, `csv`. This function *creates* and *uses* the network client. There's no seam. The only way in for a test is patching.

Testing without a seam

```
@patch("etl.extract")
@patch("etl.transform")
@patch("etl.load")
def test_main(mock_load, mock_transform, mock_extract,
              source_data, transformed_data):
    mock_extract.return_value = source_data
    mock_transform.return_value = transformed_data

    main()

    mock_load.assert_called_once_with(transformed_data, "users_transformed")


def test_extract(monkeypatch, source_data):
    mock_s3_client = MagicMock()
    mock_s3_client.download_fileobj.return_value = source_data

    mock_file = MagicMock()
    mock_file.__enter__.return_value = source_data

    monkeypatch.setattr("boto3.client", lambda service: mock_s3_client)
```

Five patches. The test knows about `boto3`, `csv`, `builtins.open`, tempfiles. Switch `download_fileobj` to `get_object` and the test breaks even though behavior didn't.

> "Patch is a sign of failure. The more you have to patch, the worse your code is."
>
> Michael Foord, creator of `unittest.mock` (talk · 35:02)

Without DI vs. with DI

Without DI

```
def extract():
    s3 = boto3.client("s3")
    bucket = os.environ["AWS_BUCKET"]
    return _read_from(s3, bucket, "users.csv")
```

Creates and uses the client. No seam.

With DI

```
def extract(reader):
    return reader.read("users.csv")
```

Use only. Creation moved to the caller. The reader is the **seam**.

Worked example: ETL with DI

```python
from storage import Reader, Writer  # ABCs: .read(key) / .write(key, rows)

class ETL:
    def __init__(self, reader: Reader, writer: Writer):
        self.reader = reader
        self.writer = writer

    def extract(self, key):     return self.reader.read(key)
    def transform(self, rows):  ...    # pure
    def load(self, rows, key):  self.writer.write(key, rows)

    def run(self, src, dst):
        rows = self.extract(src)
        self.load(self.transform(rows), dst)
```

`reader` and `writer` are the seams. The `Reader` / `Writer` ABCs in `storage.py` declare the contract. Concrete implementations inherit and live there too: `S3Reader`, `LocalFileReader`, `PostgresWriter`, `SqliteWriter`. Tests wire in fakes or in-memory SQLite.

> Heads-up: substituting fakes / in-memory I/O isn't a replacement for an integration test against a real local Postgres. It's one strategy. We're going deep on this one; the rest live behind the blog link.

The fakes

```python
class FakeReader:
    """In-memory stand-in. Maps a logical name to canned rows."""
    def __init__(self, rows_by_name):
        self.rows_by_name = rows_by_name
        self.reads = []

    def read(self, name):
        self.reads.append(name)
        return self.rows_by_name[name]


def test_extract_with_fake():
    reader = FakeReader({"users": SOURCE_ROWS})
    etl = ETL(reader=reader, writer=None)

    assert etl.extract("users") == SOURCE_ROWS
    assert reader.reads == ["users"]
```

The fake exposes the same `.read(key)` method as `S3Reader`. The test runs without `boto3`, the network, or any patching. Compare to the patch-stack slide. Note the key here is a *logical* name; there is no file. With `LocalFileReader` the same `read(key)` call would treat the key as a filename under `base_path`.

Exercise 3.1 · Inject and test

12 min

Open: `modules/3.1_dependency_injection/`

You get:

- `storage.py` : `Reader` / `Writer` ABCs + concrete subclasses.
- `etl.py` : `ETL(reader, writer)` typed against the ABCs.
- `test_etl_START.py` : `FakeReader` , `test_extract_with_fake` , `test_extract_with_local_file` (uses `LocalFileReader` + `tmp_path`), and `test_transform` already wired up.

You write:

- `FakeWriter` : a recording fake. Implement `__init__` and `write` .
- `test_load` . Pick one: **(A)** use your `FakeWriter` , or **(B)** use `SqliteWriter` from `storage.py` with `sqlite3.connect(":memory:")` .
- Bonus: `test_run` wiring through `ETL.run` .

Run:

```
pytest modules/3.1_dependency_injection -q
```

TDD: red, green, refactor, and TDD-lite

Strict red-green-refactor lands well when:

- The spec is sharp.
- The behavior change is small.
- You already understand the code.

When the work is exploratory, **"tests soon" beats "tests first."** Run code by hand, learn what it does, then write tests as understanding solidifies. The goal is testable design, and strict TDD is one path to that goal.

Aside: there is also a **top-down vs bottom-up** axis to how you grow a feature. Ask in Q&A if curious.

Hands-on follow-up: `modules/side_quests/tdd_cycle/` walks a full red-green-refactor on FizzBuzz (~10 min).

Snapshot / approval testing

Question for the group: can you test code that you don't understand?

Discuss.

Snapshot / approval testing

When you don't fully understand a function and need to change it without breaking things:

- **Detective:** read carefully, work out what it *should* do, write tests for that. Slow. Highest confidence.
- **Snapshot:** call it, capture the output, lock that in as the baseline. Fast. Catches drift but encodes current behavior including bugs.

Common advice: "test behavior, not implementation." Snapshot tests accept *more* brittleness on purpose, so every change shows up in the diff. Use snapshots while you're still learning the code; replace them with behavior-driven tests once you understand it.

POINTER

- approvaltests: fuller-featured library on PyPI. We're rolling our own to keep the moving parts visible.

Exercise 3.2 · Snapshot test

7 min

Open: `modules/3.2_snapshot_testing/`

`compute_invoice` has tier discounts, bulk discounts, a coupon, and a weird tax-exempt branch. You don't have to read it.

1. Run the worksheet as a script. It prints current outputs.
2. Paste the lines into `EXPECTED`.
3. `pytest modules/3.2_snapshot_testing -q` should be green.
4. Modify `gnarly.py` and watch tests fail loudly.

Brainstorm callback: "Got a function in your code nobody trusts?" Next week, see if you can work in a snapshot test.

Advanced: how can this technique be combined with fuzzing or property-based testing?

Where to go next

Attendee survey · 5 min

TODO

Conference survey details TBD.

CI · GitHub Actions · 2 min

```yaml
# .github/workflows/tests.yml
name: Tests
on:
  push:
    branches: [main]
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -e ".[dev]"
      - run: pytest
```

- Runs `pytest` on every PR and on every push to `main`
- The YAML is plumbing and boilerplate
- Whether you use GH Actions or not, the value in Continuous Integration is in the feedback loop. Every change runs the suite without anyone remembering to.
- High ROI automation when you're working in a shared repo. Set it up for your team if you don't have it already.

Agentic testing · self-correct loop · 1 min

- A **test harness** is runner + tests + the green/red signal -> coding agent's automated feedback loop.
- Combine with automated linting, formatting (ruff), and type checking (pyright, mypy).
- Tests give the agent a contract to verify against. It reads failures and iterates.
- You can lean on tests for verification only as far as the tests are good. Still have to read them and understand them.
- Example: Claude Superpowers plugin ships testing-flavored skills.
  - ⚠️ FYI: AI tooling moves fast; treat this skill demo as a snapshot
  - If you're new to Software Engineering (SWE) practices, try reading this skill

```
$ /skills
superpowers:test-driven-development
superpowers:verification-before-completion
superpowers:systematic-debugging
superpowers:writing-plans
superpowers:executing-plans
superpowers:requesting-code-review
superpowers:receiving-code-review
... 14 skills total
```

Can AI write all my tests?

2 min

Maybe? 🤷 Please check and read them.

Example of AI-generated test code. Let's critique.

Agent-written

```python
def test_users_endpoint():
    result = client.get("/users").json()
    # So many unnecessary asserts
    # What is `result` supposed to look like?!
    assert result["users"][0]["id"] == 42
    assert result["users"][0]["name"] == "alice"
    assert result["users"][1]["status"] == "active"
    assert len(result["users"]) == 3
    assert result["meta"]["count"] == 3
```

Human-written (after one debugger pass)

```python
def test_users_endpoint():
  expected = {"users": [
    {"id": 42, "name": "alice", "status": "active"},
    {"id": 7,  "name": "bob",   "status": "active"},
    {"id": 13, "name": "carol", "status": "pending"},],
    "meta": {"count": 3},
    }
    result = client.get("/users").json()
    assert result == expected
```

- [5/12/26]: I see these kinds of tests every other day using the latest state-of-the-art models.
- When I do code reviews and see tests like this, I downweight my confidence in the level of human review absent other verification.
- Theory: agents inspect objects less than a human with debugger, so the multi-assertion shape is the tell.
- When I use coding agents and get code like on the left, I use that as a starting point. Set a breakpoint in the System Under Test (SUT). Extracting the full expected result is usually cleaner and gives higher verification of entire result object.

Where to go next · 3 min

Topics worth exploring after today.

- **Mocking depth:** `monkeypatch` vs `mock.patch`, decorator vs `with`.
- **Coverage:** `pytest-cov`, branch coverage.
- **Property-based:** `hypothesis`.
- **Plugins:** `pytest-randomly` (catch order dependence), `pytest-xdist` (parallel).
- **DataFrame testing:** `pandas.testing.assert_frame_equal`.
- **Debugging tests:** VS Code debugger, `pdb`, `breakpoint()`, `capsys` / `capfd`.
- **Design principles:** SOLID, Law of Demeter, composition over inheritance.
- **CI:** see Appendix for a minimal GitHub Actions workflow.

What will you test first?

Back to the project you brainstormed at the start.

- One function or method that's been bugging you.
- One bug you'd like to lock down with a regression test.
- One untested module you'd like to put a snapshot test around.

Q&A

Thanks for spending the afternoon with me.

paulzuradzki.com

Appendix · Resources

Articles (paulzuradzki.com)

- Dependency injection for Pythonistas
- Unit testing SQL
- Python packaging resources

Reference docs

- pytest documentation
- Python module search path
- hypothesis · property-based testing